DEPARTMENT 1 · DEVELOPMENT AND IT · PROFESSION 8

E-commerce developer

Profession 8 of 10 in Development and IT.

What the profession means

An e-commerce developer builds the parts of an online shop that handle money and stock: product data, carts, discounts, shipping rules, payments, orders and integrations with warehouses and accounting. Some use hosted platforms (Shopify, WooCommerce); others build custom shops.

What professionals do every day

Models products and prices, writes cart and checkout logic, connects payment and shipping providers, syncs orders and stock, tests money calculations carefully, and monitors failed payments.

Why businesses hire them, and what problems they solve

A shop that gets money calculations wrong loses customers and trust. A developer makes sure totals, discounts, taxes and delivery rules are correct and reliable.

- Wrong totals and discounts at checkout.
- Manual order and stock work between channels.
- Failed or duplicated payments.

Skills you need

- Money handling: store amounts as whole paise or cents, never floating-point errors.
- Carts, discounts and shipping rules.
- Payment gateways and webhooks (for example Razorpay, Stripe).
- Order, stock and accounting integrations.
- Security (PCI scope), testing and monitoring.

Tools (free and paid)

Tool	What it is for	Free option	Paid option	Official link (not checked live)
Shopify	Hosted e-commerce platform	Trial and development stores	Paid plans	www.shopify.com
WooCommerce	E-commerce plugin for WordPress	Free plugin	Hosting and extensions cost	woocommerce.com
Stripe	Payment processing API	Per-transaction fees (check current pricing)	Per-transaction fees	stripe.com/docs
Razorpay	Payment gateway for India	Test mode	Per-transaction fees	razorpay.com/docs
Node.js test runner	Built-in tests (node:test)	Free	Free	nodejs.org/api/test.html

Learning sequence: beginner, intermediate, advanced

Level	What you can do when you finish
Beginner	Product catalogue and a checkout page; test totals by hand.
Intermediate	Coupons, shipping rules, payment test mode and order emails.
Advanced	Webhooks, stock sync, reporting and a multi-channel integration.

Projects by level

- **Beginner:** A small shop with a cart and one payment method in test mode.
- **Intermediate:** A coupon and shipping engine with automated tests.
- **Advanced:** An integration that syncs orders to accounting software.

Client-ready portfolio project

A checkout for a fictional or permissioned shop: cart, coupon, shipping rules, payment in test mode, order confirmation, and an automated test suite for money calculations.

Worked example

Worked example. A basket of Rs 180 × 2 plus Rs 90 is Rs 450. A 10% coupon takes Rs 45 off, leaving Rs 405. Below the Rs 500 threshold, Rs 49 shipping applies, so the total is Rs 454. The tests in the solution check these numbers.

Assignment with full solution

ASSIGNMENT

Cart totals with coupons and free delivery

Business problem: A bakery's online shop must show the right total before checkout: items, a coupon (percent or flat), and free delivery above a threshold.

Steps

1. Create `cart.js` and `test.js` as shown.
2. Run `node test.js`. All three checks must pass.
3. Add one new test: a flat coupon larger than the subtotal must not give a negative total.
4. Explain the rule you used for the free-delivery threshold, and whether it compares before or after the discount.

Expected output: Passing tests and a one-paragraph explanation of the business rules.

SOLUTION (ANSWER KEY)

Why it works. Each function does one job, so each can be tested alone. Shipping is decided after the discount (the comparison uses `afterDiscount`), which is a business choice the shop must confirm. The flat coupon is capped with `Math.min`, so the discount can never exceed the subtotal: a flat Rs 600 coupon on a Rs 450 basket gives a discount of Rs 450, leaving a subtotal of 0, so

shipping of Rs 49 applies and the total is Rs 49. **Test added:** `assert.strictEqual(cartTotal([{price:100,qty:1}], {type:'flat', value:500}, rule).total, 49)`, which holds because the discount is capped at the subtotal and the Rs 49 fee applies below the threshold. Tax is not included; add it only after checking current rules.

JAVASCRIPT

```
// cart.js : pure functions for a cart total (easy to test)
function lineTotal(item) {
  return item.price * item.qty;
}
function applyCoupon(subtotal, coupon) {
  if (!coupon) return 0;
  if (coupon.type === 'percent') return Math.round(subtotal * coupon.value / 100);
  if (coupon.type === 'flat') return Math.min(coupon.value, subtotal);
  return 0;
}
function cartTotal(items, coupon, shippingRule) {
  const subtotal = items.reduce((sum, item) => sum + lineTotal(item), 0);
  const discount = applyCoupon(subtotal, coupon);
  const afterDiscount = subtotal - discount;
  const shipping = afterDiscount >= shippingRule.freeAbove ? 0 : shippingRule.fee;
  return { subtotal, discount, shipping, total: afterDiscount + shipping };
}
module.exports = { lineTotal, applyCoupon, cartTotal };

// test.js : run with `node test.js`
const assert = require('assert');
const { cartTotal } = require('./cart');
const rule = { freeAbove: 500, fee: 49 };
const basket = [{ price: 180, qty: 2 }, { price: 90, qty: 1 }]; // subtotal 450
assert.deepStrictEqual(cartTotal(basket, null, rule), { subtotal: 450, discount: 0, shipping:
49, total: 499 });
assert.deepStrictEqual(cartTotal(basket, { type: 'percent', value: 10 }, rule),
  { subtotal: 450, discount: 45, shipping: 49, total: 454 });
assert.deepStrictEqual(cartTotal([{ price: 300, qty: 2 }], null, rule),
  { subtotal: 600, discount: 0, shipping: 0, total: 600 });
console.log('all cart tests passed');
```

Common mistakes

- Calculating money with floating-point numbers, which can drift by a paisa.
- Trusting the browser for prices; always recalculate on the server.
- Testing payments in live mode.
- Missing webhook handling, so a paid order never confirms.

Freelancing opportunities

- Shop builds on Shopify and WooCommerce.
- Payment and shipping integrations.
- Freelance briefs for 'checkout', 'Shopify app', 'payment gateway integration', 'order sync'.

Learning resources

LEARNING RESOURCES (VERIFY BEFORE USE)

- Stripe documentation
- Razorpay documentation
- Node.js built-in test runner

Part of **Master Freelancing: Development and IT**. Back to the chapter list on skillnavora.in · Open the full textbook. Official links in this chapter were not checked live; confirm before relying on them.